Evaluating Fast Matrix Multiplication Algorithms In Rust

Computational Laboratory Project Presentation

Author:

Alberto Defendi

Supervisor:

Leonardo Robol

Introduction

- **Matrix Multiplication:** One of the most fundamental computations in numerical linear algebra and scientific computing.
- **Fast Algorithms:** Perform asymptotically fewer floating-point operations and require less data communication than the classical $O(N^3)$ algorithm.
- **Project Goals:**
- Implement fast matrix multiplication using arbitrary CP tensor decompositions in Rust.
- Evaluate native Rust linear algebra library (`faer`) against optimized vendor GEMMs (Intel `MKL dgemm` via FFI).
- Handle general dimensions via memory-efficient **dynamic peeling** instead of zero-padding.
- Benchmark sequential and parallel execution using multiple scheduling strategies.

Tensors & Outer Products

Definition: 3-Way Tensor

A 3-dimensional (or 3-way) tensor is a multidimensional array $T \in \mathbb{R}^{M \times N \times P}$. Its slices obtained by fixing one index are matrices. For example, the k-th frontal slice of T is the matrix X_k.

Definition: Outer Product

For vectors $u \in \mathbb{R}^M, v \in \mathbb{R}^N, w \in \mathbb{R}^P$, the rank-1 outer product tensor $T = u \circ v \circ w \in \mathbb{R}^{M \times N \times P}$ has entries:

$$x_{ijk} = u_i v_j w_k$$

The rank of a tensor T is the minimum number of rank-1 tensors that generate T as their sum.

Low-Rank Tensor Decompositions

Definition: CP Decomposition

The CANDECOMP/PARAFAC (CP) decomposition of rank R of a tensor T is:

$$T = [\![U, V, W]\!] = \sum_{r=1}^{R} u_r \circ v_r \circ w_r$$

where $U \in \mathbb{R}^{M \times R}$, $V \in \mathbb{R}^{N \times R}$, and $W \in \mathbb{R}^{P \times R}$ are factor matrices with columns u_r, v_r, and w_r.

Definition: Mode-k Product

The mode-k product between a tensor $T \in \mathbb{R}^{N_1 \times N_2 \times N_3}$ and a matrix $U \in \mathbb{R}^{R \times N_k}$ for $k \in \{1, 2, 3\}$ is denoted by $Y = T \times_k U$. Its matricized representation is:

$$Y_{(k)} = U T_{(k)}$$

Bilinear Forms via Low-Rank Decompositions

- A bilinear form $f : X \times Y \to Z$ is represented by a tensor T:

$$f = T \times_1 x^T \times_2 y^T$$

- Using a CP decomposition $T = [\![U,V,W]\!]$ of rank R:

$$f(x,y) = \sum_{l=1}^{R} (u_l^T x) \cdot (v_l^T y) w_l$$

- Evaluating $f(x,y)$ requires exactly R active scalar multiplications and $O(R)$ additions.
- For matrix multiplication $C = AB$, representing the form $\langle M,N,P \rangle$, we have:

$$\mathrm{vec}(C^T) = T \times_1 \mathrm{vec}(A)^T \times_2 \mathrm{vec}(B)^T$$

- Given a CP decomposition of $T = [\![U,V,W]\!]$, we get:

$$\mathrm{vec}(C^T) = \sum_{l=1}^{R} (s_l \cdot t_l) w_l$$

where $s_l = u_l^T \mathrm{vec}(A)$ and $t_l = v_l^T \mathrm{vec}(B)$.

Strassen’s Algorithm & CP Decomposition

- **Strassen’s Algorithm** for $\langle 2,2,2 \rangle$ uses $R = 7$ multiplications and 18 additions:

$$U = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & -1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & -1 \end{pmatrix},$$

$$V = \begin{pmatrix} 1 & 1 & 0 & -1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & -1 & 0 & 1 & 0 & 1 \end{pmatrix},$$

$$W = \begin{pmatrix} 1 & 0 & 0 & 1 & -1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & -1 & 1 & 1 & 0 & 0 & 0 \end{pmatrix}$$

- This reduces arithmetic complexity from $O(N^3)$ to $O(N^{\log_2 7}) = O(N^{2.81})$.
- Fast algorithms can be constructed for general base cases $\langle m,n,p \rangle$ using CP decompositions of rank R.

Summary of Fast Matrix Algorithms

Algorithm Base Case	Multiplications (R)	Classical (mnp)	Speedup per Step
$\langle 2,2,2 \rangle$ (Strassen)	7	8	14%
$\langle 2,2,3 \rangle$	11	12	9%
$\langle 2,2,4 \rangle$	14	16	14%
$\langle 2,2,5 \rangle$	18	20	11%
$\langle 2,3,3 \rangle$	15	18	20%
$\langle 2,3,4 \rangle$	20	24	20%
$\langle 2,4,4 \rangle$	26	32	23%
$\langle 3,3,3 \rangle$	23	26	17%
$\langle 3,3,4 \rangle$	29	36	24%
$\langle 3,3,6 \rangle$	40	54	35%

Dynamic Peeling

- Traditional fast algorithms require dimensions to match the base case power.
- **Dynamic Peeling:** A memory-efficient alternative to zero-padding.
- Split A and B into core components and peeled boundary vectors/scalars:

$$A = \begin{pmatrix} A_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}, q \quad B = \begin{pmatrix} B_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}$$

- Recombine using a combination of the core product and boundary corrections:

$$C = \begin{pmatrix} C_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix} = \begin{pmatrix} A_{11}B_{11} + a_{12}b_{21} & A_{11}b_{12} + a_{12}b_{22} \\ a_{21}B_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{pmatrix}$$

- Leaf-node $A_{11}B_{11}$ is solved recursively, while all boundary corrections are consolidated into single, large GEMM calls to maximize cache locality and vendor BLAS efficiency.

Shared Memory Parallel Scheduling

To balance computational load and avoid recursion tree overhead, we benchmarked three scheduling strategies in Rust using the Rayon library:

- **DFS (Depth-First-Search):**
- Evaluates the recursion tree sequentially.
- Uses all available threads inside vendor leaf GEMM calls.
- High thread under-utilization during split and addition phases.
- **BFS (Breadth-First-Search):**
- Parallelizes all independent recursive subproblems at the top levels.
- Rayon executes recursive branches in parallel.
- **Hybrid:**
- Compensates for BFS load imbalance.
- BFS task parallelism is applied to the first $R^L - (R^L \bmod P)$ tasks (where P is the thread count).
- DFS (all threads on single GEMM) is used on the remaining $R^L \bmod P$ tasks.

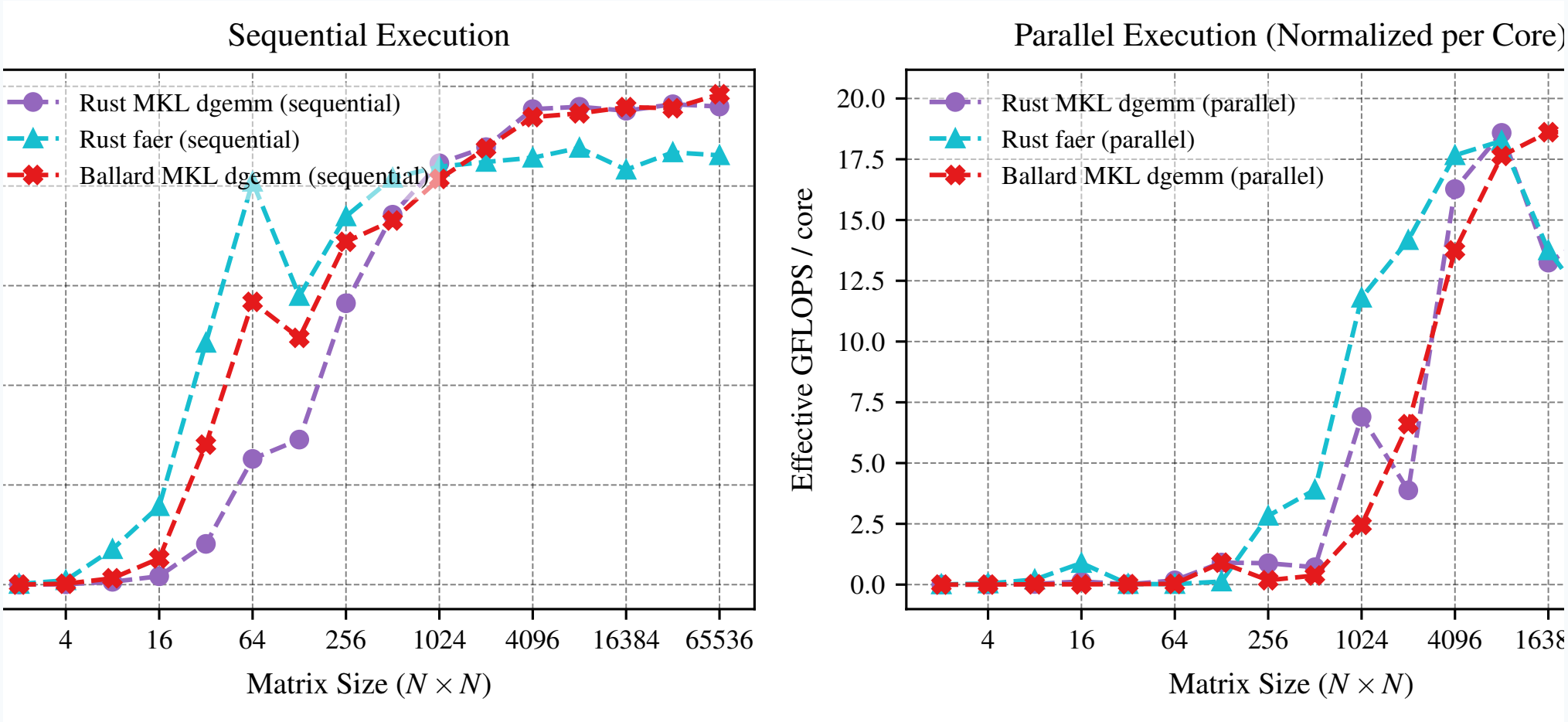
Execution Platform & Setup

- **Hardware:** One node of a dual-socket AMD EPYC 7763 server (64 physical cores, 2 TB RAM).
- **Software environment:**
- GCC 13.2.0, Intel OneAPI compilers 2025.0.4, Intel OneAPI MKL.
- Rust Nightly (2026-06-23) utilizing LLVM 22 for optimized AVX2 and FMA generation.
- **Linear Algebra Backends:**
- **Intel MKL dgemm:** Vendor library called via custom FFI wrapper.
- **faer matmul:** Native Rust high-performance linear algebra library.
- **Metric:**

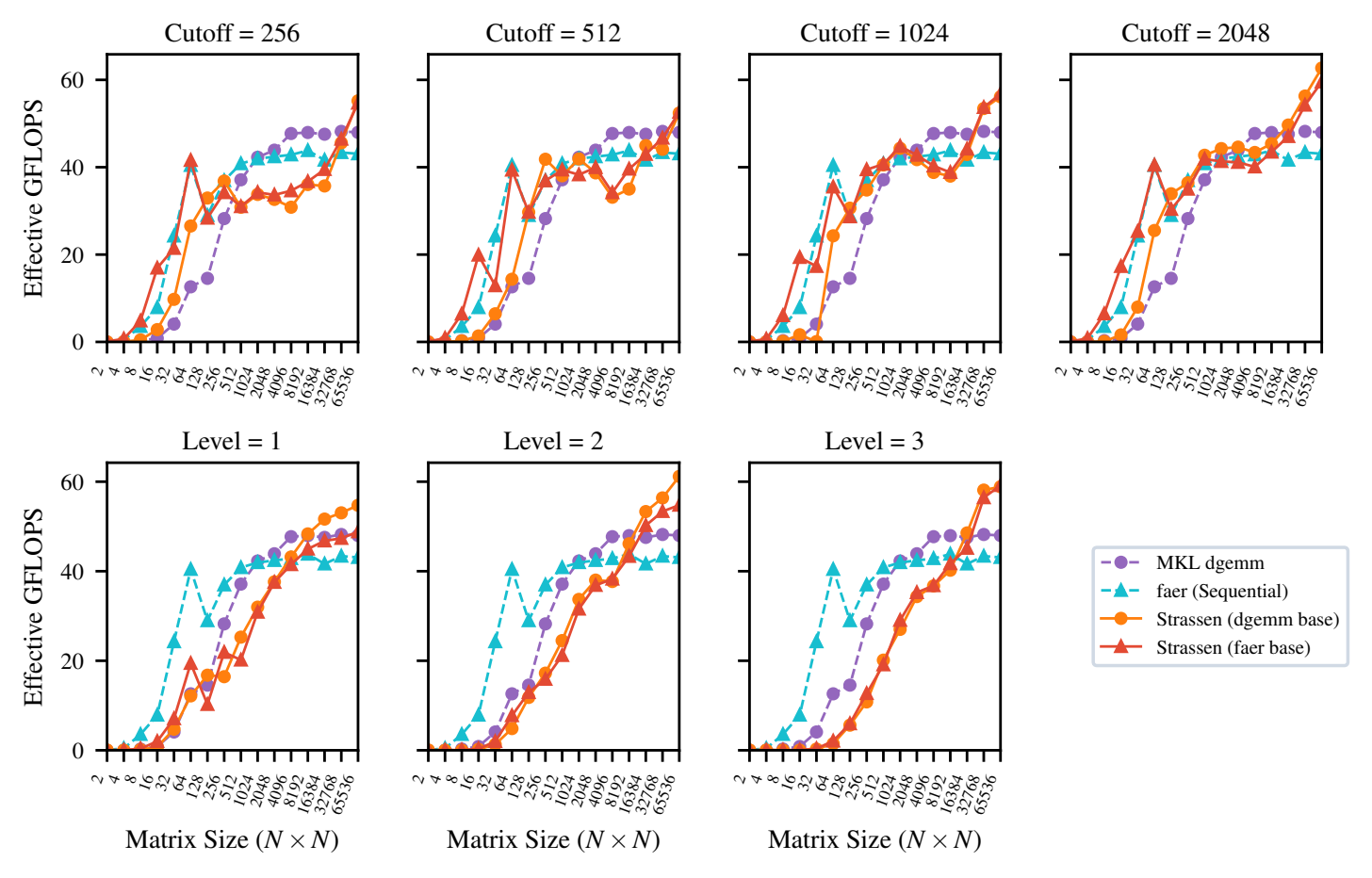
$$\text{Effective GFLOPS} = \frac{2MNP - MP}{\text{time in seconds} \cdot 10^9}$$

which normalizes performance based on the classical floating-point operation count.

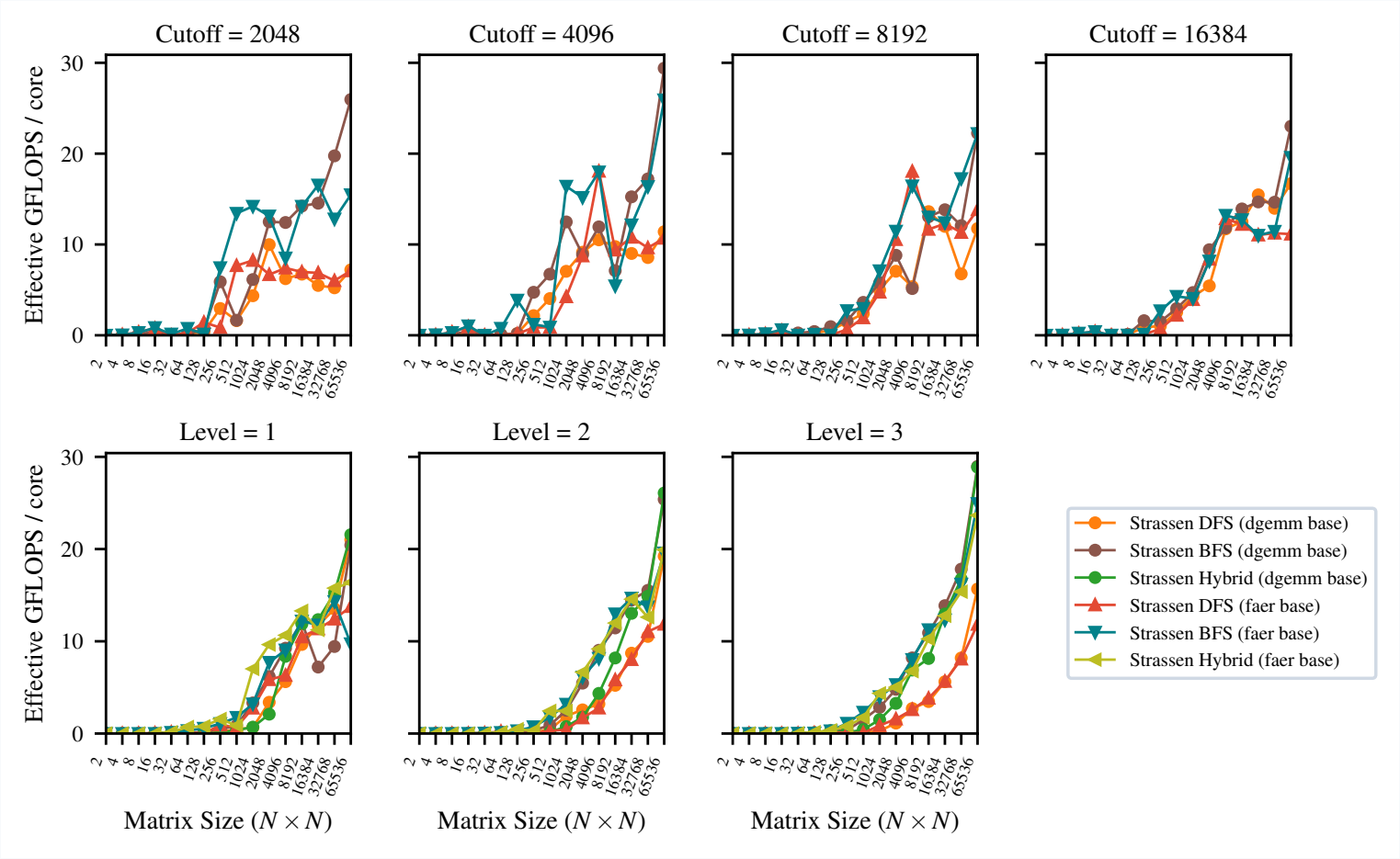
Performance of Base GEMM Libraries



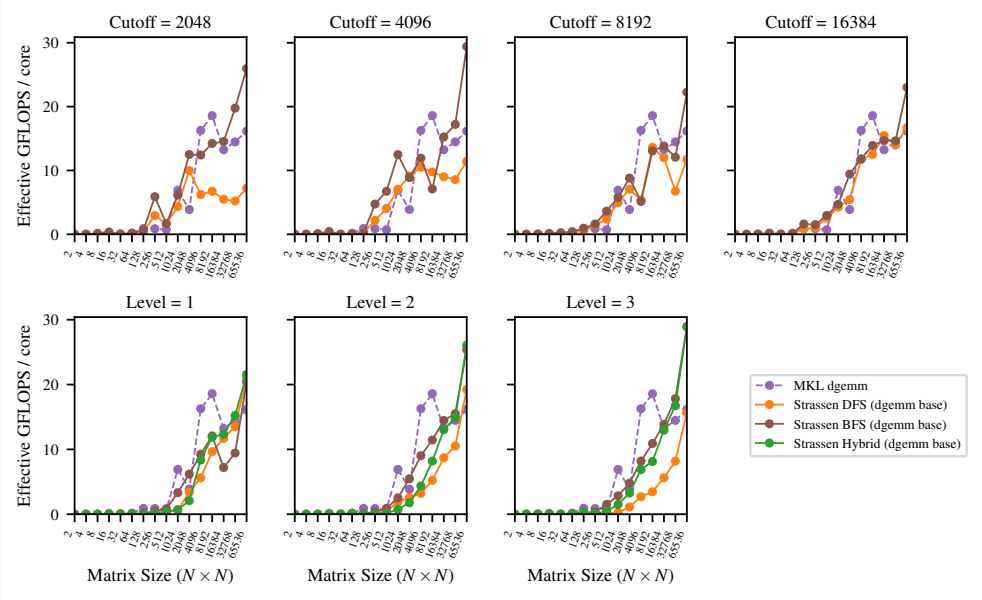
Sequential Crossover Point

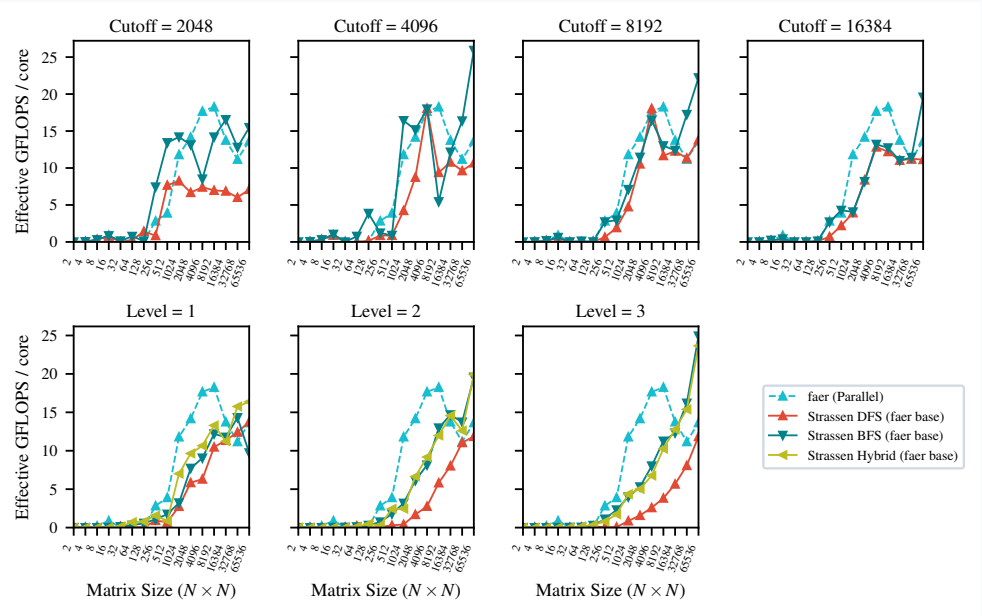


Parallel Crossover (Normalized per Core)

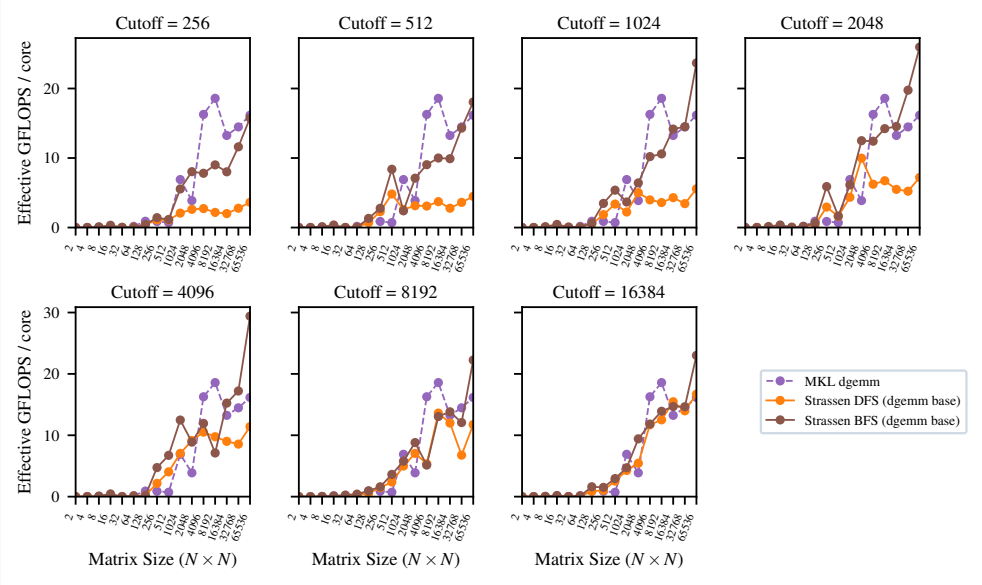


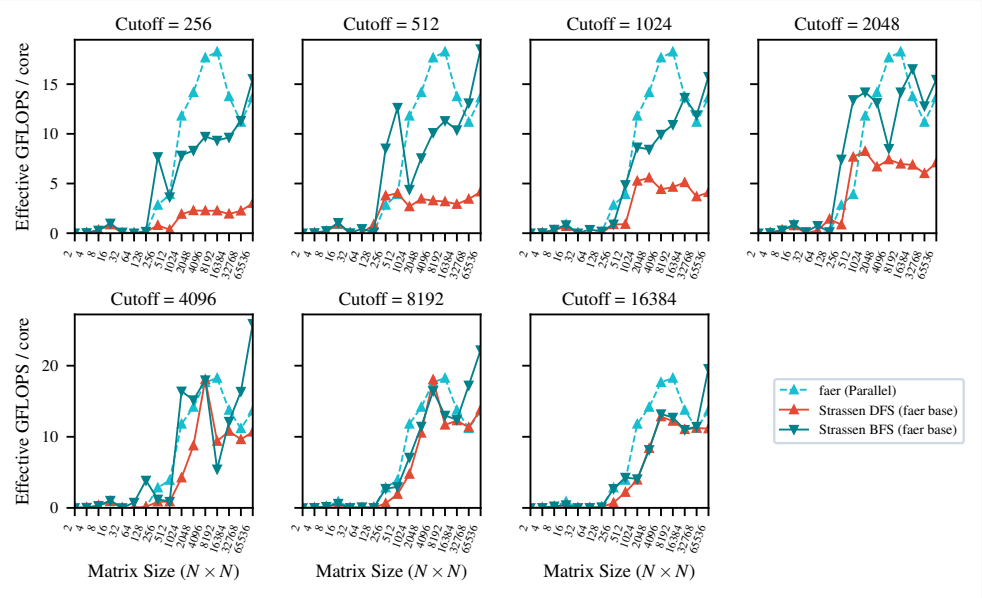
Parallel Cutoffs: MKL vs. Faer Bases



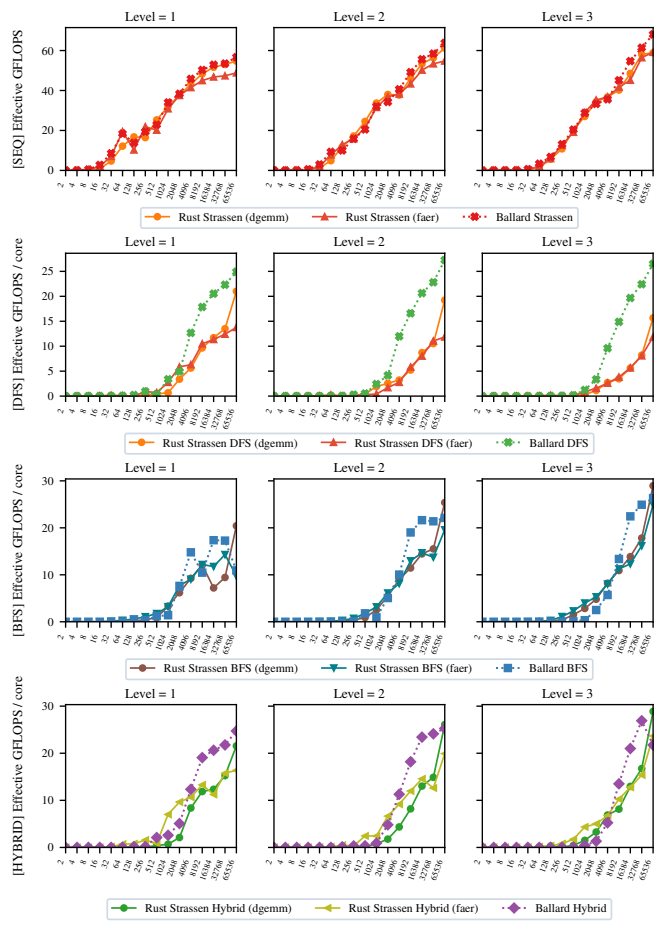


Finding the Optimal Size Cutoff





Comparison with Benson & Ballard C++ Reference



Conclusions & Future Work

Key Takeaways:

- Fast matrix multiplication algorithms successfully outperform optimized vendor/native GEMM libraries in both sequential and parallel execution.
- Crossover points are highly practical ($N = 8192$ sequential, $N = 16384$ parallel).
- Rayon's work-stealing scheduler provides superior load-balancing and scaling compared to OpenMP at deeper levels of recursion.

- **Future Directions:**
- **Profiling & SIMD:** Finer refactoring of matrix split/addition routines using SIMD vectorization.
- **Other GEMM Backends:** Add support for OpenBLAS, BLIS, or other open-source libraries.
- **Distributed/GPU version:** Port to GPU (CUDA/WebGPU) or distributed clusters to scale beyond $N = 65536$.
- **Architectural Comparisons:** Evaluate on high-core-count Intel Xeon processors.